

Imam Abdulrahman Bin Faisal University

College of Computer Science and Information Technology
Department of Computer Engineering

ARTI403 - Image Processing Lab 2 Report

Image Sampling, Quantization, Arithmetic and Set/Logical Operations

Student Name	Mohammed Moayed Alquriniy
Student ID	2240007325
Academic Year	2025 - 2026
GitHub Repository	github.com/itsMo33/ARTI403---Image-Processing

1. Objective

The objective of this lab is to study the effect of sampling and quantization on an image, and to apply arithmetic operations (addition, subtraction, adding a constant) and set/logical operations (union, intersection, difference, symmetric difference) on images.

2. Tools

Python 3, Jupyter Notebook, and the libraries OpenCV, NumPy, Pillow, scikit-image, and Matplotlib.

3. Implementation and Results

```
import os
import numpy as np
from PIL import Image
import cv2

os.makedirs("../images", exist_ok=True)

if not os.path.exists("../images/lena_gray_256.tif"):
    from skimage import data
    cam = data.camera()
    Image.fromarray(cam).save("../images/cameraman.tif")

    astro = data.astronaut()
    astro_gray = cv2.cvtColor(astro, cv2.COLOR_RGB2GRAY)
    astro_gray = cv2.resize(astro_gray, (256, 256), interpolation=cv2.INTER_AREA)
    Image.fromarray(astro_gray).save("../images/lena_gray_256.tif")

    size = 400
    A = np.zeros((size, size), dtype=np.uint8)
    cv2.circle(A, (170, 200), 120, 255, -1)
    B = np.zeros((size, size), dtype=np.uint8)
    cv2.rectangle(B, (150, 100), (380, 330), 255, -1)
    Image.fromarray(A).save("../images/A.png")
    Image.fromarray(B).save("../images/B.png")

    print("Sample images generated in ../images/")
else:
    print("Using existing images in ../images/")
```

Output:
Using existing images in ../images/

Observation: The images already exist in the shared images folder, so the existing images were used and no new images were generated.

1) Import the libraries

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
```

2) Define Functions

```
def sample_image(image, factor):
    """
    Downsamples the image by the given factor.
    Args:
        image (numpy array): Original image.
        factor (int): Factor by which to downsample.
    Returns:
        numpy array: Downsampled image.
    """
    height, width = image.shape[:2]
    sampled_image = cv2.resize(image, (width // factor, height // factor),
                               interpolation=cv2.INTER_NEAREST)
    return sampled_image

def quantize_image(image, levels):
    """
    Reduces the number of grayscale levels in the image.
    Args:
        image (numpy array): Original image.
        levels (int): Number of grayscale levels.
    Returns:
        numpy array: Quantized image.
    """
    quantized_image = np.floor(image / (256 // levels)) * (256 // levels)
    quantized_image = quantized_image.astype(np.uint8)
    return quantized_image

def plot_images(original, sampled, quantized):
    """
    Plots the original, sampled, and quantized images side by side.
    """
    plt.figure(figsize=(12, 4))
    plt.subplot(1, 3, 1)
    plt.imshow(original, cmap='gray')
    plt.title('Original Image')
    plt.axis('off')

    plt.subplot(1, 3, 2)
    plt.imshow(sampled, cmap='gray')
    plt.title('Sampled Image')
    plt.axis('off')

    plt.subplot(1, 3, 3)
    plt.imshow(quantized, cmap='gray')
    plt.title('Quantized Image')
    plt.axis('off')
    plt.show()
```

3) Import the image and set the parameters

```
image_path = '../images/lena_gray_256.tif'
sampling_factor = 14
quantization_levels = 9

# Load image in grayscale
original_image = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
if original_image is None:
    print(f"Error: Unable to load image at {image_path}")

# Sample and quantize
sampled_image = sample_image(original_image, sampling_factor)
quantized_image = quantize_image(original_image, quantization_levels)

# Plot results
plot_images(original_image, sampled_image, quantized_image)
```

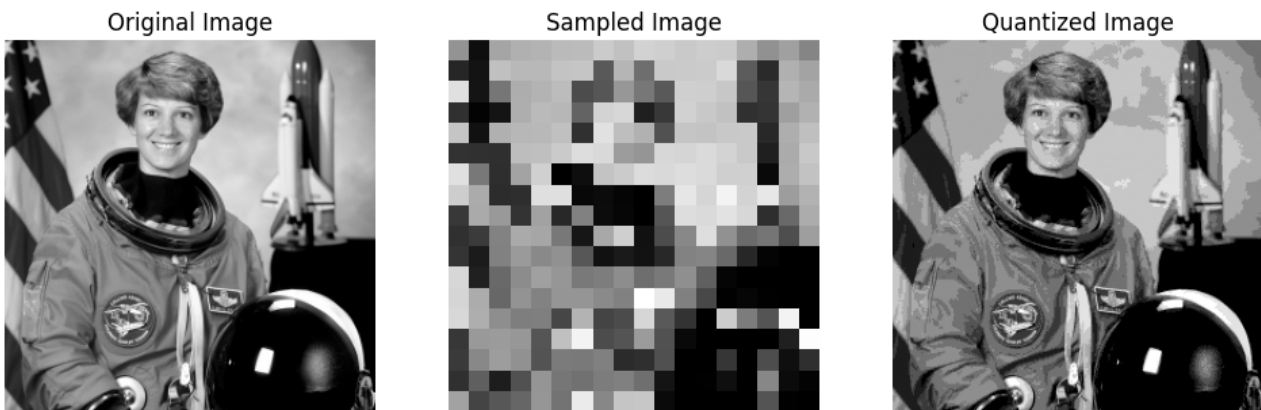


Figure 1

Observation: With a sampling factor of 14 the 256×256 image becomes only 18×18 pixels, so it looks very blocky (pixelated). With 9 quantization levels the image is still clear, but small bands of gray appear in the smooth areas.

Arithmetic Operations

```
from PIL import Image

img1 = Image.open('../images/lena_gray_256.tif')
img2 = Image.open('../images/cameraman.tif')

resize = (400, 400)
img1 = img1.resize(resize, Image.Resampling.LANCZOS)
img2 = img2.resize(resize, Image.Resampling.LANCZOS)

im1arr = np.asarray(img1)
im2arr = np.asarray(img2)

addition = im1arr.astype(np.int16) + im2arr.astype(np.int16)
addition = np.clip(addition, 0, 255).astype(np.uint8)
resultImage = Image.fromarray(addition)
plt.imshow(resultImage, cmap='gray'); plt.title('Addition'); plt.axis('off'); plt.show()
```



Figure 2

Observation: Adding the two images overlaps their content. Many sums are larger than 255 and are clipped to 255, so the result is very bright and large areas become white.

Sets and Logical Operations

```
img3 = Image.open('../images/A.png')
img4 = Image.open('../images/B.png')

img3 = img3.resize(resize, Image.Resampling.LANCZOS)
img4 = img4.resize(resize, Image.Resampling.LANCZOS)

im3arr = np.asarray(img3)
im4arr = np.asarray(img4)

union = im4arr | im3arr
resultImage2 = Image.fromarray(union)
plt.imshow(resultImage2, cmap='gray'); plt.title('Union (A OR B)'); plt.axis('off'); plt.show()
```

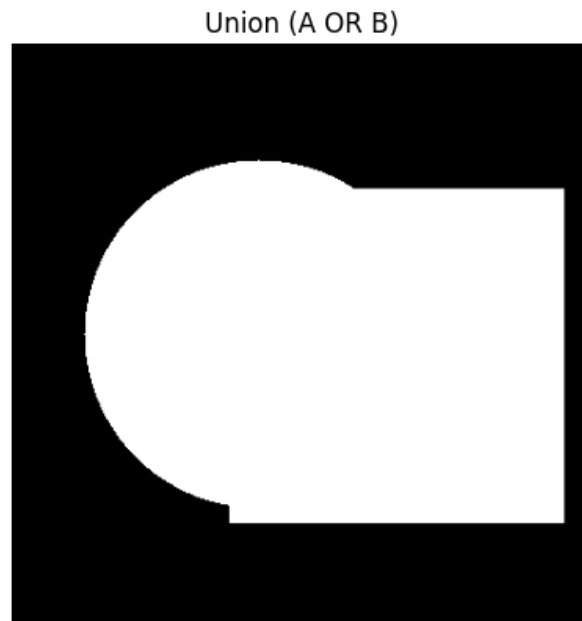


Figure 3

Observation: The union (A OR B) is white wherever a pixel is white in A or in B, so the circle and the square are combined into one shape.

Practical Session Tasks

Task #1 — Change the sampling and quantization parameters and observe the effects

```
param_sets = [
    (2, 2),    # very light sampling, only 2 gray levels (near black & white)
    (4, 4),
    (14, 9),   # value given in the manual
    (30, 32),  # heavy downsampling, fine quantization
]

fig, axes = plt.subplots(len(param_sets), 3, figsize=(12, 4 * len(param_sets)))
for row, (factor, levels) in enumerate(param_sets):
    sampled = sample_image(original_image, factor)
    quantized = quantize_image(original_image, levels)

    axes[row, 0].imshow(original_image, cmap='gray'); axes[row, 0].set_title('Original'); axes[row,
    0].axis('off')
    axes[row, 1].imshow(sampled, cmap='gray'); axes[row, 1].set_title(f'Sampled (factor={factor})');
    axes[row, 1].axis('off')
    axes[row, 2].imshow(quantized, cmap='gray'); axes[row, 2].set_title(f'Quantized (
    levels={levels})'); axes[row, 2].axis('off')

plt.tight_layout()
plt.show()
```

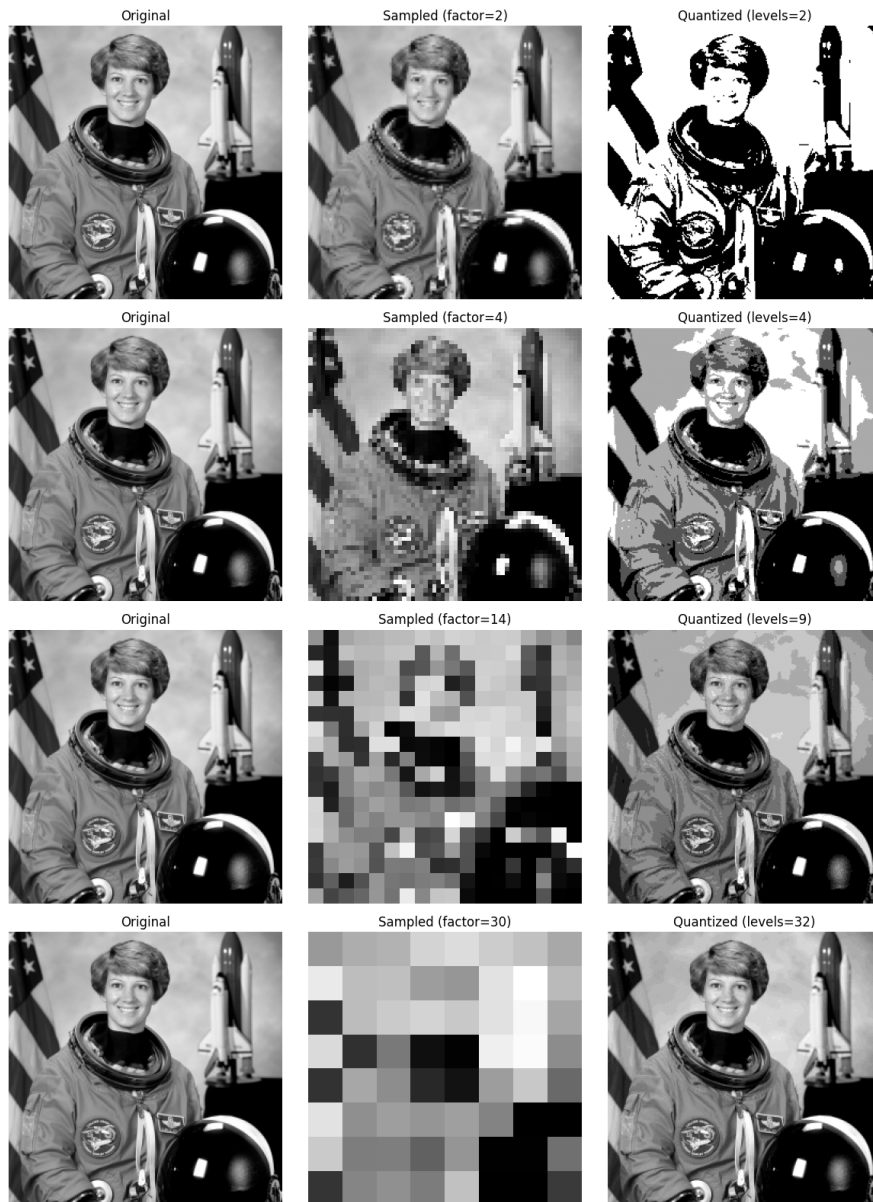


Figure 4

Observation: Sampling reduces the spatial resolution: with factors 2 and 4 the image still looks almost the same, while factors 14 and 30 produce large blocks and the details are lost. Quantization reduces the number of gray levels: 2 levels give a black and white image, 4 levels show clear false contours, 9 levels look acceptable, and 32 levels look almost the same as the original.

Task #2 — Arithmetic and set/logical operations on two images

a) Subtract two images and display the result

```
gray1 = im1arr.astype(np.int16)
gray2 = im2arr.astype(np.int16)

subtraction = np.clip(gray1 - gray2, 0, 255).astype(np.uint8)
plt.imshow(subtraction, cmap='gray'); plt.title('Subtraction (Image1 - Image2)'); plt.axis('off');
plt.show()
```



Figure 5

Observation: Negative differences are clipped to 0, so most of the result is dark. Only the areas where Image 1 is brighter than Image 2 remain visible.

b) Add one image with a constant value of 175 and display it

```
add_constant = np.clip(gray1 + 175, 0, 255).astype(np.uint8)
plt.imshow(add_constant, cmap='gray'); plt.title('Image1 + 175'); plt.axis('off'); plt.show()
```



Figure 6

Observation: Adding 175 makes the whole image much brighter. Every pixel with a value of 80 or more reaches 255 and becomes white, so only the darkest areas keep their details.

c) Set difference on two grayscale images ($A - B = A \text{ AND NOT } B$)

```
setA = im3arr
setB = im4arr

set_difference = cv2.bitwise_and(setA, cv2.bitwise_not(setB))
plt.imshow(set_difference, cmap='gray'); plt.title('Set difference (A - B)'); plt.axis('off');
plt.show()
```

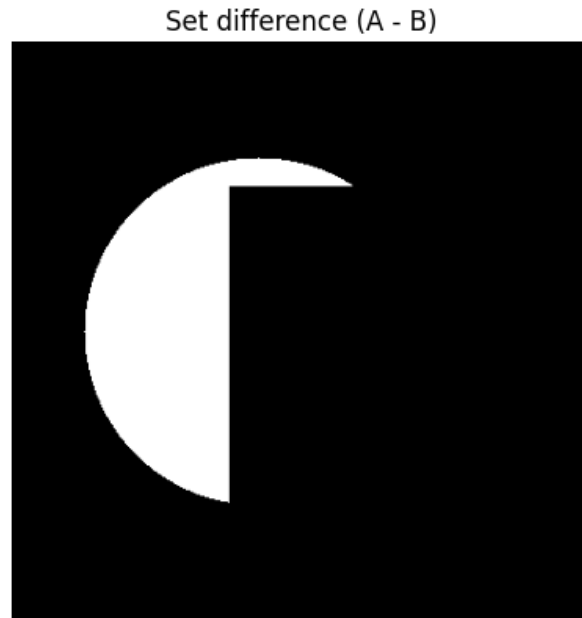


Figure 7

Observation: $A - B$ keeps only the part of the circle (A) that is outside the square (B).

d) Symmetric difference on two grayscale images (A XOR B)

```
symmetric_difference = cv2.bitwise_xor(setA, setB)
plt.imshow(symmetric_difference, cmap='gray'); plt.title('Symmetric difference (A XOR B)');
plt.axis('off'); plt.show()
```

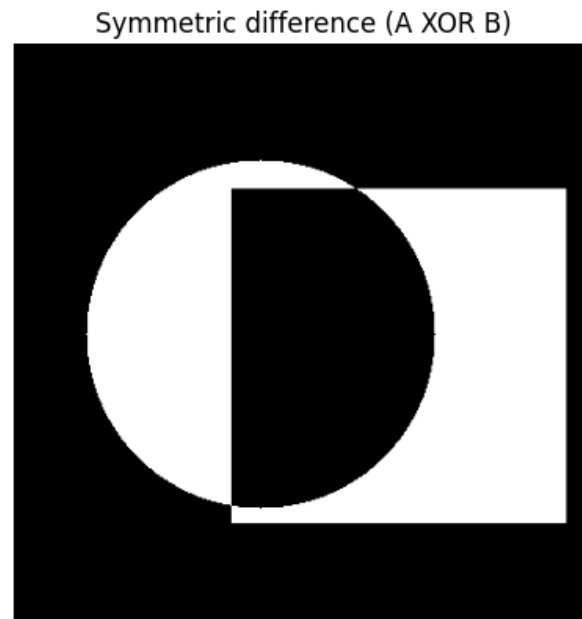


Figure 8

Observation: A XOR B keeps the areas that belong to only one of the two shapes. The overlapping area becomes black.

e) Intersection of two grayscale images (A AND B)

```
intersection = cv2.bitwise_and(setA, setB)
plt.imshow(intersection, cmap='gray'); plt.title('Intersection (A n B)'); plt.axis('off'); plt.show(
)
```

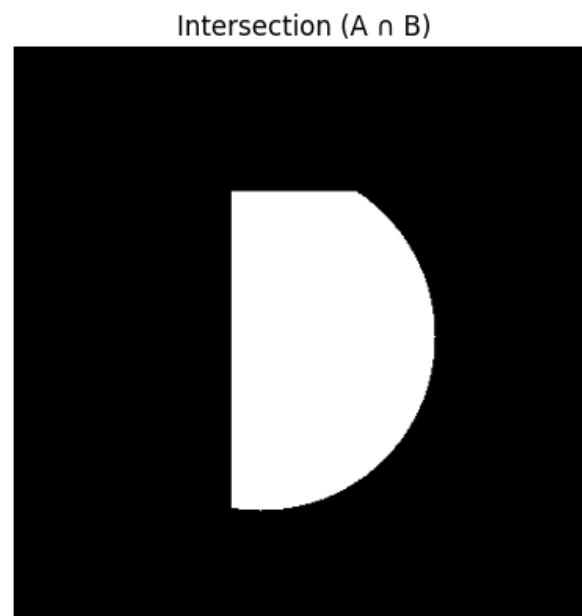


Figure 9

Observation: A AND B keeps only the overlapping area between the circle and the square.

4. Conclusion

In this lab, sampling and quantization were applied to an image. Sampling controls the spatial resolution, so large sampling factors cause pixelation, while quantization controls the number of gray levels, so few levels cause false contouring. Arithmetic operations changed the brightness of images and needed clipping to keep the values in the range 0 to 255. Finally, set operations were applied on binary images using bitwise operations to get the union, intersection, difference, and symmetric difference.